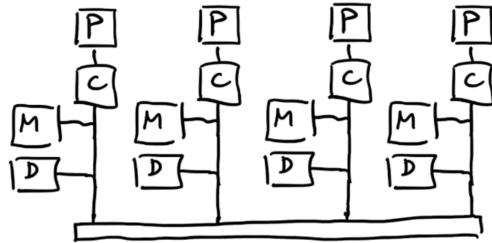

Those of you who received a bonus in the competition, mark the tasks for which you are claiming it.

1. Sketch a NUMA shared-memory system and point out its main advantages over a UMA system.



Advantages:

- *Memory bandwidth scales with the number of nodes: if processors are mainly accessing their local memory, there is no single shared bus that becomes a bottleneck.*
- *Lower latency to local data.*
- *Better scalability to a larger number of cores.*

2. A computation applies three element-wise functions to an array $\mathbf{a}$, one after another: $\mathbf{b} = f(\mathbf{a})$, then $\mathbf{c} = g(\mathbf{b})$, then $\mathbf{d} = h(\mathbf{c})$. Which parallel pattern can we use for the computation? How can we improve its performance on a shared-memory system, and what is the reasoning behind the improvement?

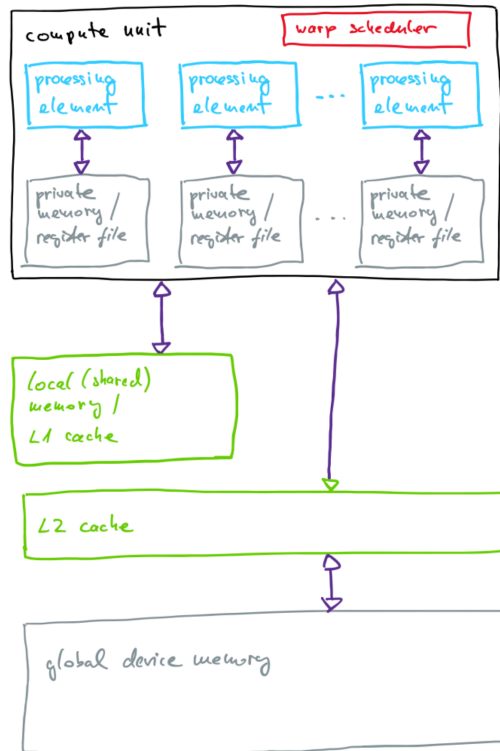
It is the map pattern (a sequence of three map operations).

Improvement: fuse the three maps into a single map, i.e. for each element compute $h(g(f(a[i])))$ in one pass, reducing the number of memory operations.

3. The program below renders an image: each pixel (i, j) of an $n \times n$ grid is computed independently, but the number of iterations inside compute varies strongly from pixel to pixel. Parallelise it using OpenMP to run efficiently for any image size and thread count.

```
#pragma omp parallel for collapse(2) schedule(dynamic)
for (int i = 0; i < n; i++) {
    for (int j = 0; j < n; j++) {
        image[i][j] = pixelCompute(i, j);
    }
}
```

4. Sketch the memory hierarchy of a GPU as seen by a single compute unit. Indicate which threads can access each type of memory and give a rough order-of-magnitude access time of each.



register file: 1 period
local memory: 30 periods
global memory: 500 periods

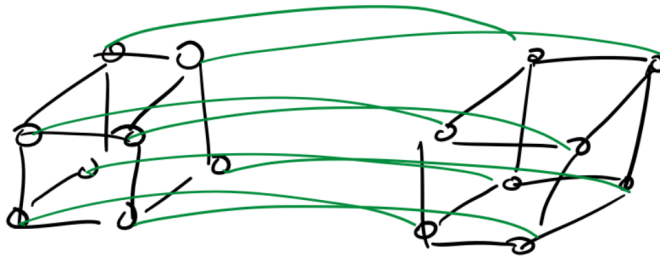
5. Describe how a warp is executed when its threads take different branches of an if-then-else statement. How does this reflect on performance?

All threads of a warp share one instruction stream. On a divergent branch the warp executes both paths serially: the then-branch runs with the else-threads masked off, then the else-branch runs with the then-threads masked off. Divergence wastes throughput (worst case, the cost of both paths combined).

6. The CUDA kernel below transposes an $N \times N$ matrix. Rewrite the kernel for better performance by taking care of coalesced memory access. 256 threads in a block are organised in a 2D.

```
__global__ void transpose(float *out, float *in, int N) {
    __shared__ float tile[16][16+1];
    int x = blockIdx.x * blockDim.x + threadIdx.x;
    int y = blockIdx.y * blockDim.y + threadIdx.y;
    if (x < N && y < N)
        tile[threadIdx.y][threadIdx.x] = in[y * N + x];
    __syncthreads();
    if (x < N && y < N)
        out[y * N + x] = tile[threadIdx.x][threadIdx.y];
}
```

7. One option to connect compute nodes is a 4-dimensional hypercube. Sketch it and state its number of nodes, the number of links per node, and the maximum number of hops between any two nodes. Hint: use binary representation for compute node addressing.



number of nodes: $2^4 = 16$

links per node: 4

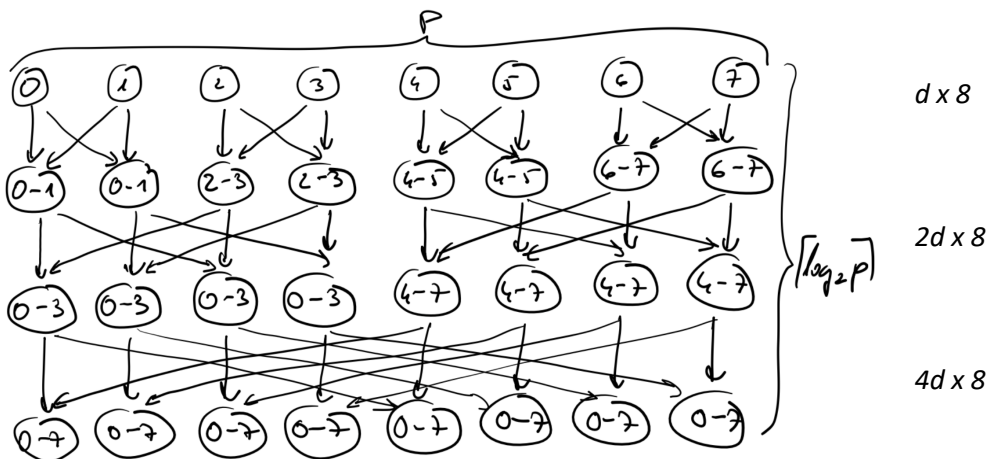
number of hops equals the Hamming distance

maximal number of hops: 4

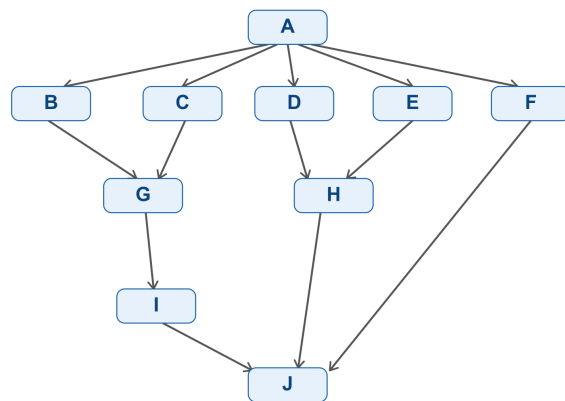
8. In an iterative MPI simulation, every process computes a local error each step, and all processes must agree on the global maximum error before deciding whether to continue. Check the code and, if necessary, correct it.

```
double errGlobal;
int stop = 0;
while (!stop) {
    double errLocal = runSimulation();
    MPI_Allreduce(&errLocal, &errGlobal, 1, MPI_DOUBLE,
                 MPI_MAX, 0, MPI_COMM_WORLD);
    stop = (errGlobal < TOL);
}
```

9. Design an efficient communication scheme in which every process collects the data held by all other processes. Assume the distributed-systems model in which a process can communicate with only one other process per time step. Assume 8 processes, each holding d data elements. Indicate the quantity of data transferred between processes at each communication step.



-
10. The direct acyclic graph shown below describes a parallel computation dependency. Each of 10 tasks may start only once all its predecessors have finished. Determine the maximal speedup of the computation using Amdahl's law and the work-span model. Comment the results.



$$t_1 (\text{work}) = 10$$

$$t_\infty (\text{span}) = 5 \text{ (longest } A \rightarrow B/C \rightarrow G \rightarrow I \rightarrow J \text{)}$$

$$\text{work-span model: } S \leq \frac{t_1}{t_\infty} = 2$$

Amdahl's law

- $f = \frac{2}{10}$ as only A and J cannot run in parallel with any other process
 - with $p = \infty$: $S \leq \frac{1}{f} = 5$
-